Deep Delta Hedging

Lewis Wilcher

September 2026

Abstract

This piece of work compares the results of classic quantitative finance in pricing and hedging vanilla European derivatives with what has been coined 'Deep Hedging' by more modern literature such as Buehler et al. [1]. We look at increasingly complex hedging problems, starting from simple Black-Scholes delta hedging with no frictions, and moving to Whalley and Wilmott's [6] optimal hedging result under proportional transaction costs. In each case a feedforward neural network is trained directly on simulated paths of the underlying, with no knowledge of the closed form solution, and its performance is compared against the corresponding analytical benchmark. In the frictionless setting the network recovers a hedging strategy close in both shape and aggregate performance to the Black-Scholes delta, providing an empirical demonstration of Hornik's universal approximation theorem [4] in a case where the target function is known exactly. Under transaction costs, where the network and the Whalley-Wilmott band are trained and evaluated against a common entropic risk objective, the network is found to achieve a lower entropic risk than both the naive delta hedge and the closed form band at a fixed, discrete rebalancing frequency. These results suggest that deep hedging is a genuine competitor to classical analytical hedging strategies, even in this simplified constant-volatility setting.

1 Frictionless Hedging of a European Call Option

1.1 Classic Black-Scholes Hedging

For our analytical solution to this problem we look to the results of the Black-Scholes paper [2] and Merton [3] who both derive the Black-Scholes PDE and give the answer of delta hedging in order to hedge against the risk of the underlying movements.

1.1.1 Market and Contract

We work in the standard Black-Scholes market. The underlying S_t follows a geometric Brownian motion under the real world measure,

$$dS_t = \mu S_t \, dt + \sigma S_t \, dW_t,$$

where μ is the real world drift, σ the volatility, and W_t a standard Brownian motion. Cash earns the risk free rate r, so a bond evolves as $dB_t = rB_t \, dt$. We sell a European call with strike K and maturity T, with payoff at maturity

$$\Phi(S_T) = (S_T - K)^+.$$

Throughout we assume no transaction costs, the effect of relaxing this assumption is considered in a later section.

1.1.2 The Replicating Portfolio and the PDE

The seller's task is to construct a self-financing portfolio of the underlying and cash that replicates $\Phi(S_T)$ exactly, so that the risk of having sold the option is eliminated. Let $C(t, S)$ denote the value of the option at time t for a given stock price S, and consider a portfolio holding Δ_t units of the underlying and the remainder in cash. By Itô's formula,

$$dC = \left(\frac{\partial C}{\partial t} + \mu S \frac{\partial C}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 C}{\partial S^2} \right) dt + \sigma S \frac{\partial C}{\partial S} \, dW_t.$$

Choosing $\Delta_t = \partial C / \partial S$ eliminates the random term dW_t from the hedged portfolio, since the exposure of the option to the stock is exactly cancelled by holding Δ_t shares. The portfolio is then instantaneously riskless, and by a no-arbitrage argument it must earn the risk free rate. This yields the Black-Scholes PDE,

$$\frac{\partial C}{\partial t} + rS \frac{\partial C}{\partial S} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 C}{\partial S^2} - rC = 0, \qquad C(T, S) = (S - K)^+,$$

where the real world drift μ has dropped out entirely: the hedge, and hence the price, does not depend on it. This is the essential feature of complete markets that the deep hedging literature departs from once frictions are introduced.

1.1.3 The Delta Hedging Strategy

Solving the PDE gives the closed form Black-Scholes call price,

$$C(t,S) = S\,\Phi(d_1) - Ke^{-r(T-t)}\Phi(d_2),$$

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}}, \qquad d_2 = d_1 - \sigma\sqrt{T-t},$$

where Φ denotes the standard normal CDF. The replicating position derived above is the option's delta,

$$\Delta(t,S) = \frac{\partial C}{\partial S} = \Phi(d_1).$$

The classic hedging strategy is therefore: at every point in time, hold $\Delta(t,S_t)$ shares of the underlying, funded by the premium received and rebalanced continuously as S_t and t evolve. In continuous time and with no frictions this strategy replicates the option payoff exactly, so the hedging error is identically zero.

1.1.4 Discrete Implementation

In practice the position can only be rebalanced at a finite set of times $t_0, \ldots, t_N = T$, here $N = 100$ equally spaced dates over a one year maturity, with $S_0 = K = 100$, $r = 0.05$ and $\sigma = 0.2$. At each date t_i the position is reset to $\Delta(t_i, S_{t_i})$, funded from a cash account earning r, and any imbalance between the old and new position is bought or sold at the prevailing price. The hedging error is measured as the difference, at maturity, between the value of this replicating portfolio and the option payoff owed:

$$\text{PnL} = \Big(\text{cash and shares at } T\Big) - (S_T - K)^+.$$

Because N is finite rather than continuous, a small residual hedging error remains even under the Black-Scholes strategy. This error shrinks as N grows and is the benchmark against which the network's performance is measured in the next section.

1.2 Neural Network Hedging

The delta hedging result of the previous section rests on the existence of a closed form solution to the Black-Scholes PDE. This is a consequence of the market's simplicity: log-normal dynamics, constant volatility, and no frictions. Once these assumptions are relaxed, for instance by introducing proportional transaction costs, stochastic volatility, or discrete rebalancing constraints, a closed form for the optimal hedge is in general unavailable, and the problem must instead be solved numerically.

The approach taken here follows the deep hedging framework of Buehler et al. [1], in which the hedging strategy itself is represented as a parametric function,

a feedforward neural network, and its parameters are learned by minimising an appropriate objective over simulated paths of the underlying, rather than solved for analytically. The justification for representing an unknown hedging strategy in this way rests on the universal approximation theorem of Hornik [4], which shows that a feedforward network with as few as a single hidden layer, arbitrary width, and a sufficiently general (nonconstant, bounded) activation function can approximate any continuous function on a compact domain to arbitrary accuracy. Since the true optimal hedging strategy at each rebalancing date is a function of the observable market state, in the classical case $\delta^*(t, S) = \Phi(d_1(t, S))$, this result provides the theoretical grounding for using a network as a function approximator for the hedge ratio itself, without knowing δ^* in closed form: the network need not be told the Black-Scholes formula, only trained so that its output, as a function of the same information, reproduces the behaviour of a good hedge. Section 2 is treated as an exercise in this fact: because the true optimal strategy is known in closed form in the frictionless case, it is possible to verify directly whether the network is able to recover it from data alone.

1.2.1 Network Architecture and Inputs

The hedging strategy is represented by a feedforward network $f_\theta : \mathbb{R}^5 \to \mathbb{R}$, with two hidden layers of width 32 and hyperbolic tangent activations,

$$h^{(1)} = \tanh(W^{(1)}x+b^{(1)}), \qquad h^{(2)} = \tanh(W^{(2)}h^{(1)}+b^{(2)}), \qquad f_\theta(x) = W^{(3)}h^{(2)}+b^{(3)},$$

where θ collects all weights and biases. At each re-balancing date t_i the network receives as input the state

$$x_i = \Big(S_{t_i}/K,\ (S_{t_i} - K)/K,\ \tau_i,\ r,\ \sigma\Big), \qquad \tau_i = T - t_i,$$

and the position taken is

$$\delta_i = \Phi_{\text{sig}}\big(f_\theta(x_i)\big), \qquad \Phi_{\text{sig}}(z) = \frac{1}{1 + e^{-z}}.$$

Two departures from a naive implementation of the network are worth explaining, since both follow directly from properties of the target being approximated.

First, the previous position δ_{i-1}, though a natural candidate input for a general hedging problem, is deliberately excluded here. In the frictionless setting the optimal hedge at each date depends only on (t, S), so a rational hedger with no cost of trading always resets to $\delta^*(t, S)$ regardless of what was held a moment before; the previous position carries no information relevant to the optimal decision. Including it would add a spurious feedback channel, in which the network's own earlier output re-enters as an input at the next date, coupling errors across time steps without providing any signal the target function actually depends on. This channel becomes relevant once transaction costs are introduced, and is reinstated at that point.

Second, the raw output of the final linear layer is passed through a sigmoid, so that $\delta_i \in (0, 1)$ by construction. This is not an arbitrary choice of output

nonlinearity: the true target $\delta^*(t, S) = \Phi(d_1(t, S))$ is itself a standard normal CDF applied to an affine function of $\ln S$, and so is bounded in $(0, 1)$ for any call. Building this constraint into the architecture is a form of inductive bias, it restricts the network's hypothesis space to functions with the same range as the true solution, rather than requiring the training procedure to discover the boundedness property purely from data.

The same network, with the same parameters θ, is applied at every rebalancing date, so the full hedging strategy is a single function of five variables evaluated N times along each path. Notably, the Black-Scholes delta is not supplied to the network at any point; the purpose of the exercise is to establish whether it can be recovered purely from training on simulated outcomes.

1.2.2 Training

The network is trained on simulated paths of the same geometric Brownian motion as in Section 2, with real world drift $\mu = 0.10$, over $N = 100$ rebalancing dates. For a batch of n_{paths} simulated trajectories, the hedging portfolio is constructed exactly as in the discrete implementation above, but with the position at each date chosen by the network rather than the closed form delta, and the resulting hedging error $\text{PnL}(\theta)$ recorded at maturity for each path. The network is trained to minimise the mean squared hedging error over the batch,

$$L(\theta) = \frac{1}{n_{\text{paths}}} \sum_{j=1}^{n_{\text{paths}}} \text{PnL}_j(\theta)^2,$$

using the Adam optimiser [5] with an initial learning rate of 10^{-3}, decayed to zero over training by a cosine annealing schedule, over 3500 iterations, with a fresh batch of 1000 simulated paths drawn at each iteration. Because $\text{PnL}(\theta)$ depends on the network's output at every one of the $N = 100$ rebalancing dates, gradients of the loss with respect to θ are propagated back through the full chain of rebalancing decisions at each iteration, analogous to backpropagation through time in a recurrent network, even though, with δ_{i-1} removed from the input, the decisions themselves are no longer directly coupled through a feedback loop.

1.2.3 Evaluation

Once trained, the network's hedging performance is compared against the closed form Black-Scholes delta strategy of Section 2, on a common set of freshly simulated test paths not seen during training. Both strategies are evaluated using the same hedging error and cost structure, with the comparison summarised by the mean and standard deviation of PnL across the test set. Since the frictionless problem is solved exactly by the Black-Scholes delta, this comparison serves as validation: if the network's parameters have converged appropriately, its hedging performance should approach that of the closed form strategy, providing an empirical demonstration of Hornik's theorem in a setting where the target function is explicitly known, and establishing the network as a viable candidate

for the setting where transaction costs are introduced in the following section, and no such closed form target is available.

In addition to the aggregate PnL statistics, the learned strategy is compared directly against the Black-Scholes delta as a function of the state. At a fixed rebalancing date, the network's position on each test path is plotted against the corresponding stock price and overlaid on the closed form curve $\Phi(d_1(t, S))$, giving a direct visual check of whether the network has recovered the correct functional shape across the range of S actually visited during simulation, including in the sparsely sampled tails, rather than relying solely on the aggregate hedging error to judge convergence.

1.2.4 Results

Table 1 reports the mean and standard deviation of the hedging error at maturity, over 1000 test paths, for both strategies.

	PnL Mean	PnL Std
Black-Scholes delta	0.0266	0.6347
Neural network	0.0229	0.7289

Table 1: Hedging error statistics under the Black-Scholes delta strategy and the trained network, on a common set of 1000 test paths, $N = 100$ rebalancing dates.

The two strategies achieve a comparable mean hedging error, both close to zero relative to the scale of the option, consistent with neither strategy holding a systematic bias in the direction of the residual error. The Black-Scholes delta does not achieve a mean of exactly zero either, which reflects the fact that even the closed form strategy can only be rebalanced at $N = 100$ discrete dates rather than continuously; some residual hedging error is therefore unavoidable under either strategy at this rebalancing frequency, and the comparison should be read as network against discretely-rebalanced Black-Scholes, not against a perfect continuous hedge.

The standard deviation of the network's hedging error is modestly higher than that of the Black-Scholes strategy, at a ratio of approximately 1.15. Figure 1 examines this gap directly by comparing the learned position against the closed form delta as a function of S, at the midpoint of the hedging horizon. The network's positions lie closely along the Black-Scholes curve across the bulk of the distribution of S, including into the tails, which is consistent with the network having recovered the correct functional shape of the hedge rather than a systematically biased approximation to it. The residual gap in standard deviation is accordingly attributable to the network's positions departing from the exact curve by a small amount on a path-by-path basis, rather than to any visible distortion in the general shape of the learned strategy, and is the expected signature of an approximation obtained by a finite amount of stochastic gradient training rather than a closed form solution.

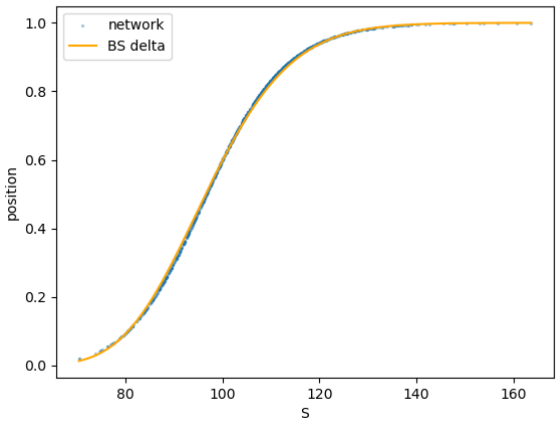


Figure 1: Network position against the Black-Scholes delta as a function of the stock price S, at t corresponding to the midpoint of the hedging horizon ($\tau = 0.5$), across 1000 test paths.

Taken together, these results are consistent with Hornik's theorem in the setting where it applies directly: the network, given only the observable state and no knowledge of the Black-Scholes formula, recovers a hedging strategy close in both shape and aggregate performance to the analytically optimal one. This provides the empirical grounding for extending the same approach, in the following section, to a setting with proportional transaction costs, where no closed form benchmark of this kind is available.

1.2.5 Code

The following is the code used for the results given in this section:

```
import math
import torch
import torch.nn as nn

# parameters, same as before
S0 = 100       # starting stock price
K = 100        # strike
r = 0.05       # risk free rate
sigma = 0.2    # volatility
T = 1.0        # maturity in years
mu = 0.10      # real world drift of the stock
n_steps = 100  # number of hedge times

# training settings
batch_size = 1000
```

```
n_iterations = 3500
learning_rate = 1e-3

torch.manual_seed(0)
normal = torch.distributions.Normal(0.0, 1.0)


def d1(S, tau):
    return (torch.log(S / K) + (r + 0.5 * sigma**2) * tau) /
        (sigma * math.sqrt(tau))


def bs_call_price(S, tau):
    a = d1(S, tau)
    b = a - sigma * math.sqrt(tau)
    return S * normal.cdf(a) - K * math.exp(-r * tau) *
        normal.cdf(b)


def bs_call_delta(S, tau):
    return normal.cdf(d1(S, tau))


# premium for the call we sell, fixed at the BS price
premium = bs_call_price(torch.tensor(float(S0)), T).item()


def simulate_paths(n_paths):
    dt = T / n_steps
    Z = torch.randn(n_paths, n_steps)

    # exact gbm step for the log of the stock
    log_returns = (mu - 0.5 * sigma**2) * dt + sigma * math.
        sqrt(dt) * Z
    S = S0 * torch.exp(torch.cumsum(log_returns, dim=1))

    # put the starting price in the first column
    start = torch.full((n_paths, 1), float(S0))
    return torch.cat([start, S], dim=1)


# the network, 5 inputs and 1 output
net = nn.Sequential(
    nn.Linear(5, 32), nn.Tanh(),
    nn.Linear(32,32), nn.Tanh(),
    nn.Linear(32, 1),
)
optimizer = torch.optim.Adam(net.parameters(), lr=
    learning_rate)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
```

```
    optimizer, T_max=n_iterations)


def nn_policy(S, prev_delta, tau):
    ones = torch.ones_like(S)
    x = torch.stack([S / K, (S - K) / K, tau * ones, r *
        ones, sigma * ones], dim=1)
    return torch.sigmoid(net(x).squeeze(1))


def bs_policy(S, prev_delta, tau):
    # only used to compare against, the network never sees
        this
    return bs_call_delta(S, tau)


def hedge_pnl(S, policy):
    n_paths = S.shape[0]
    dt = T / n_steps

    # sell the call and receive the premium
    delta = torch.zeros(n_paths)
    cash = torch.full((n_paths,), premium)
    all_deltas = []

    for i in range(n_steps):
        tau = T - i * dt

        # the policy chooses the new position
        new_delta = policy(S[:, i], delta, tau)

        # new position
        cash = cash - (new_delta - delta) * S[:, i]
        delta = new_delta
        all_deltas.append(delta)

        # cash earns interest over one step
        cash = cash * math.exp(r * dt)

    # value of the hedge portfolio at maturity
    portfolio = cash + delta * S[:, -1]

    # what we owe on the call we sold
    payoff = torch.clamp(S[:, -1] - K, min=0)

    # hedging error, zero would be a perfect hedge
    return portfolio - payoff, torch.stack(all_deltas, dim
        =1)

print("call price", premium)
```

```
108
109 for it in range(n_iterations):
110     # fresh monte carlo paths every iteration
111     S = simulate_paths(batch_size)
112
113     # hedge with the network and get the final pnl
114     pnl, _ = hedge_pnl(S, nn_policy)
115
116     # loss is the mean of pnl squared
117     loss = (pnl ** 2).mean()
118
119     # backprop then adam updates the weights
120     optimizer.zero_grad()
121     loss.backward()
122     optimizer.step()
123     scheduler.step()
124
125     if it % 250 == 0:
126         print("iteration", it, "loss", round(loss.item(), 4)
            )
127
128
129 # test on new paths, compare the network to the BS delta
130 with torch.no_grad():
131     S_test = simulate_paths(1000)
132     pnl_nn, deltas_nn = hedge_pnl(S_test, nn_policy)
133     pnl_bs, deltas_bs = hedge_pnl(S_test, bs_policy)
134
135 print("Black-Scholes Hedging:")
136 print("PNL Mean : ", pnl_bs.mean().item())
137 print("PNL STD : ", pnl_bs.std().item())
138 print("Neural Network Hedging:")
139 print("PNL Mean : ", pnl_nn.mean().item())
140 print("PNL STD : ", pnl_nn.std().item())
141
142 step = 50
143 plt.scatter(S_test[:, step].numpy(), deltas_nn[:, step].
    numpy(), s=2, label="network", alpha=0.3)
144
145 S_range = torch.linspace(S_test[:, step].min(), S_test[:,
    step].max(), 200)
146 tau = T - step * (T / n_steps)
147 plt.plot(S_range.numpy(), bs_call_delta(S_range, tau).numpy
    (), color="orange", label="BS delta")
148 plt.xlabel("S")
149 plt.ylabel("position")
150 plt.legend()
151 plt.savefig("scatter_check_deltaconstrained_noprevdelta.png"
    )
```

2 Hedging Under Transaction Costs

2.1 The Whalley–Wilmott Optimal Hedging Boundaries

The delta hedging strategy of the previous section assumes trading is frictionless. In practice every trade incurs a cost, and if this cost is ignored the strategy of continuously resetting to $\Delta(t, S)$ trades infinitely often in the continuous time limit, incurring unbounded cost. The problem of hedging optimally under transaction costs was studied by Whalley and Wilmott [6], building on the utility-based hedging framework of Davis, Panas and Zariphopoulou [7] and Hodges and Neuberger [8]. We summarise the headline results here; the full matched asymptotic derivation is not reproduced.

The setting is a market with proportional transaction costs at rate c on the notional traded, and a hedger with exponential utility, or equivalently one who minimises the entropic risk measure of terminal wealth W,

$$\rho(W) = \frac{1}{\gamma} \log \mathbb{E}\big[e^{-\gamma W}\big],$$

where γ is the risk aversion. Cash invariance of ρ reduces the state to time, the stock price, and the current position y, and dynamic programming yields a Hamilton-Jacobi-Bellman variational inequality with three regions: a no-trade region, in which it is optimal to hold the position unchanged, and buy and sell regions at its boundaries, in which the position is adjusted just enough to remain inside the no-trade region. This is a free boundary problem with no closed form solution in general.

For small c, Whalley and Wilmott obtain the boundaries in closed form by a matched asymptotic expansion in $c^{1/3}$. The result is a no-trade band centred on the frictionless Black-Scholes delta,

$$\big[\Delta(t,S) - H(t,S),\ \Delta(t,S) + H(t,S)\big], \qquad H(t,S) = \left(\frac{3\,c\,S\,\Gamma(t,S)^2\,e^{-r(T-t)}}{2\gamma}\right)^{1/3},$$

where $\Gamma = \partial^2 C / \partial S^2$ is the option's gamma. The hedger leaves the position unchanged while it lies inside this band, and trades back to the nearest edge the moment it is breached. The cube root scaling reflects a trade-off familiar from inventory control problems: transaction costs from rebalancing fall like $1/H$ as the band widens, while the risk cost of tolerating a position away from the frictionless target grows like H^2, and the optimal H balances these two terms.

This is the strategy used for the second benchmark in what follows: at each rebalancing date the previous position is clipped into $[\Delta - H, \Delta + H]$, computed from the closed form Δ and Γ at the current (t, S).

2.2 Changes to the Network

The network of the previous section is adapted to this setting in three respects.

First, the previous position δ_{i-1} is reinstated as an input, so the network is again a function $f_\theta : \mathbb{R}^6 \to \mathbb{R}$ of $\big(S_{t_i}/K,\ (S_{t_i} - K)/K,\ \delta_{i-1},\ \tau_i,\ r,\ \sigma\big)$, with the

position taken as $\delta_i = \Phi_{\text{sig}}(f_\theta(x_i))$ as before. With transaction costs present, the optimal decision at each date is no longer independent of the position already held, exactly the property that motivated the Whalley-Wilmott band above, so the previous position is now informative and is returned to the input set.

Second, the training objective is changed from the mean squared hedging error to the entropic risk of the terminal PnL,

$$L(\theta) = \frac{1}{\gamma} \log \left(\frac{1}{n_{\text{paths}}} \sum_{j=1}^{n_{\text{paths}}} e^{-\gamma \, \text{PnL}_j(\theta)} \right),$$

computed in a numerically stable form via the log-sum-exp identity. This is the same risk measure underlying the Whalley-Wilmott boundaries, so the network and the band strategy are optimised against a common objective and can be compared on equal terms.

Third, the real world drift is set to $\mu = r$. Under exponential utility a drift in excess of the risk free rate introduces a speculative position into the frictionless target, on top of the hedging demand; setting $\mu = r$ removes this term so that all three strategies below share the same underlying target $\Delta(t, S)$, and differ only in how they respond to the presence of costs.

The hedging portfolio construction is extended to deduct $c\, S_{t_i} \, |\delta_i - \delta_{i-1}|$ at every rebalancing date and once more at liquidation, and the network is trained with the Adam optimiser, an initial learning rate of 10^{-3} decayed by a cosine annealing schedule, a batch of 2000 simulated paths per iteration, and 7000 iterations.

2.3 Results

Three strategies are compared on a common set of 2000 freshly simulated test paths, with $c = 0.001$, $\gamma = 1.0$ and $N = 100$ rebalancing dates: always trading back to $\Delta(t, S)$ regardless of cost, the Whalley-Wilmott band, and the trained network. Table 2 reports the mean and standard deviation of the hedging PnL, together with the entropic risk each strategy achieves.

	PnL Mean	PnL Std	Entropic Risk
Always hedge to Δ	-0.4614	0.7022	0.7437
Whalley-Wilmott band	-0.2703	0.9840	0.8353
Neural network	-0.3919	0.8509	**0.7258**

Table 2: Hedging performance of the three strategies under proportional transaction costs, $c = 0.001$, $\gamma = 1.0$, on 2000 test paths with $N = 100$ rebalancing dates.

The always-hedge strategy trades at every one of the 100 rebalancing dates regardless of how small the resulting change in position, and this is reflected directly in its mean: the most negative of the three, as transaction costs are

paid unconditionally throughout. Its standard deviation is correspondingly the lowest, since the position never drifts from $\Delta(t, S)$. The band strategy trades only when the position leaves $[\Delta - H, \Delta + H]$, which substantially improves the mean relative to always hedging, but at the cost of a materially larger standard deviation, since the position is permitted to drift within the band between rebalances.

Notably, the band's improvement in mean is not sufficient to offset the increase in variance under the entropic risk measure at these parameters: its entropic risk of 0.8353 is worse than that of the always-hedge strategy. This is consistent with $H(t, S)$ being derived under a continuous-time, small-cost asymptotic expansion; at $N = 100$ discrete rebalancing dates the resulting band width need not be the one that minimises entropic risk exactly, and here it appears to tolerate more drift than is optimal at this finite rebalancing frequency.

The network achieves the lowest entropic risk of the three strategies, 0.7258, despite being given no information about the closed form target, the option's gamma, or the band structure at any point; its only inputs are the state variables listed above. This suggests the network has learned a better balance between trading costs and hedging risk than either benchmark: it does not ignore costs, as the always-hedge strategy does, and it is not limited by the continuous-time assumption built into the closed form band.

Figure 2 compares the position taken by each strategy against the stock price at the midpoint of the hedging horizon, overlaid on the Black-Scholes delta curve and the Whalley-Wilmott band edges computed at the same (t, S).

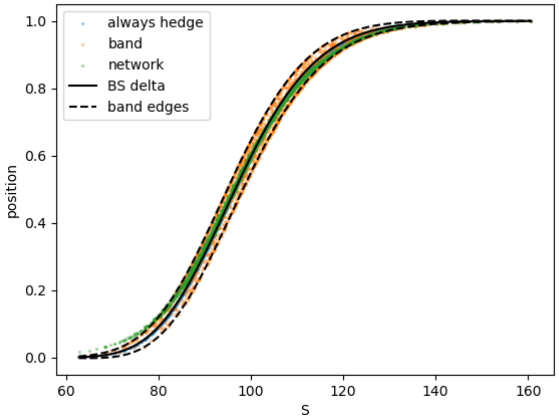


Figure 2: Position taken by each strategy against the stock price S, at $\tau = 0.5$, across 2000 test paths, overlaid on the Black-Scholes delta curve and the Whalley-Wilmott band edges.

2.4 Code

The following is the code used for the results given in this section:

```
import math
import torch
import torch.nn as nn

# parameters, same as before but mu = r
# with costs the Merton speculative position would enter every strategy
# and confuse the comparison, so mu = r removes it
S0 = 100
K = 100
r = 0.05
sigma = 0.2
T = 1.0
mu = r
n_steps = 100

# cost and risk aversion
c = 0.001       # proportional transaction cost
gamma = 1.0     # risk aversion used in the entropic loss and in the band formula

# training settings
batch_size = 2000
n_iterations = 7000
learning_rate = 1e-3

torch.manual_seed(0)
normal = torch.distributions.Normal(0.0, 1.0)


def d1(S, tau):
    return (torch.log(S / K) + (r + 0.5 * sigma**2) * tau) / (sigma * math.sqrt(tau))


def bs_call_price(S, tau):
    a = d1(S, tau)
    b = a - sigma * math.sqrt(tau)
    return S * normal.cdf(a) - K * math.exp(-r * tau) * normal.cdf(b)


def bs_call_delta(S, tau):
    return normal.cdf(d1(S, tau))


```

```
def bs_call_gamma(S, tau):
    a = d1(S, tau)
    return torch.exp(normal.log_prob(a)) / (S * sigma * math
        .sqrt(tau))


# premium for the call we sell, fixed at the BS price
premium = bs_call_price(torch.tensor(float(S0)), T).item()


def simulate_paths(n_paths):
    dt = T / n_steps
    Z = torch.randn(n_paths, n_steps)

    # exact gbm step for the log of the stock
    log_returns = (mu - 0.5 * sigma**2) * dt + sigma * math.
        sqrt(dt) * Z
    S = S0 * torch.exp(torch.cumsum(log_returns, dim=1))

    # put the starting price in the first column
    start = torch.full((n_paths, 1), float(S0))
    return torch.cat([start, S], dim=1)


# strategy 1, classic delta hedge, always trade back to the
    BS delta
def always_policy(S, prev_delta, tau):
    return bs_call_delta(S, tau)


# strategy 2, the Whalley Wilmott no trade band around the
    BS delta
def band_policy(S, prev_delta, tau):
    A = gamma * math.exp(r * tau)
    Gamma = bs_call_gamma(S, tau)
    H = (3 * c * S * Gamma**2 / (2 * A)) ** (1 / 3)

    target = bs_call_delta(S, tau)
    lower = target - H
    upper = target + H

    # clip the previous position into the band, do nothing
        if already inside
    return torch.clamp(prev_delta, min=lower, max=upper)


# strategy 3, the network, 6 inputs (incl. prev delta)
net = nn.Sequential(
    nn.Linear(6, 32), nn.Tanh(),
    nn.Linear(32, 32), nn.Tanh(),
```

```
    nn.Linear(32, 1),
)

optimizer = torch.optim.Adam(net.parameters(), lr=learning_rate)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=n_iterations)


def nn_policy(S, prev_delta, tau):
    ones = torch.ones_like(S)
    x = torch.stack([S / K, (S - K) / K, prev_delta, tau * ones, r * ones, sigma * ones], dim=1)
    return torch.sigmoid(net(x).squeeze(1))


def hedge_pnl(S, policy):
    # now includes proportional transaction costs on every trade
    n_paths = S.shape[0]
    dt = T / n_steps

    delta = torch.zeros(n_paths)
    cash = torch.full((n_paths,), premium)
    all_deltas = []

    for i in range(n_steps):
        tau = T - i * dt

        new_delta = policy(S[:, i], delta, tau)
        trade = new_delta - delta

        # pay the notional traded, plus the proportional cost of trading
        cash = cash - trade * S[:, i] - c * S[:, i] * torch.abs(trade)
        delta = new_delta
        all_deltas.append(delta)

        cash = cash * math.exp(r * dt)

    # sell the final shares, paying the cost one more time
    cash = cash - c * S[:, -1] * torch.abs(delta)

    portfolio = cash + delta * S[:, -1]
    payoff = torch.clamp(S[:, -1] - K, min=0)

    return portfolio - payoff, torch.stack(all_deltas, dim=1)

```

```
131
132 def entropic_loss(pnl, gamma):
133     # (1/gamma) log E[exp(-gamma pnl)], written with
        logsumexp for stability
134     n_paths = pnl.shape[0]
135     return torch.logsumexp(-gamma * pnl, dim=0) / gamma -
        math.log(n_paths) / gamma
136
137
138 print("call price", premium)
139
140 for it in range(n_iterations):
141     S = simulate_paths(batch_size)
142
143     pnl, _ = hedge_pnl(S, nn_policy)
144
145     # loss is now the entropic risk of the pnl, not its mean
        square
146     loss = entropic_loss(pnl, gamma)
147
148     optimizer.zero_grad()
149     loss.backward()
150     optimizer.step()
151     scheduler.step()
152
153     if it % 250 == 0:
154         print("iteration", it, "loss", round(loss.item(), 4)
            )
155
156
157 # test all three strategies on the same fresh paths
158 with torch.no_grad():
159     S_test = simulate_paths(2000)
160
161     pnl_always, deltas_always = hedge_pnl(S_test,
        always_policy)
162     pnl_band, deltas_band = hedge_pnl(S_test, band_policy)
163     pnl_nn, deltas_nn = hedge_pnl(S_test, nn_policy)
164
165     risk_always = entropic_loss(pnl_always, gamma).item()
166     risk_band = entropic_loss(pnl_band, gamma).item()
167     risk_nn = entropic_loss(pnl_nn, gamma).item()
168
169 print("Always hedge to BS delta:")
170 print("PNL mean", round(pnl_always.mean().item(), 4), "PNL
    std", round(pnl_always.std().item(), 4), "entropic risk",
     round(risk_always, 4))
171
172 print("Whalley Wilmott band:")
173 print("PNL mean", round(pnl_band.mean().item(), 4), "PNL std
```

```
    ", round(pnl_band.std().item(), 4), "entropic risk",
    round(risk_band, 4))
174
175 print("Neural network:")
176 print("PNL mean", round(pnl_nn.mean().item(), 4), "PNL std",
     round(pnl_nn.std().item(), 4), "entropic risk", round(
    risk_nn, 4))
177
178 step = 50
179 tau = T - step * (T / n_steps)
180
181 # scatter the actual chosen position of each strategy at
    this step, across all test paths
182 plt.scatter(S_test[:, step].numpy(), deltas_always[:, step].
    numpy(), s=2, label="always hedge", alpha=0.3)
183 plt.scatter(S_test[:, step].numpy(), deltas_band[:, step].
    numpy(), s=2, label="band", alpha=0.3)
184 plt.scatter(S_test[:, step].numpy(), deltas_nn[:, step].
    numpy(), s=2, label="network", alpha=0.3)
185
186 # the BS delta curve, as before, is the target that the band
     sits around
187 S_range = torch.linspace(S_test[:, step].min(), S_test[:,
    step].max(), 200)
188 target = bs_call_delta(S_range, tau)
189 plt.plot(S_range.numpy(), target.numpy(), color="black",
    label="BS delta")
190
191 # the band edges, computed the same way band_policy does,
    but as a function of S alone
192 A = gamma * math.exp(r * tau)
193 Gamma = bs_call_gamma(S_range, tau)
194 H = (3 * c * S_range * Gamma**2 / (2 * A)) ** (1 / 3)
195 plt.plot(S_range.numpy(), (target - H).numpy(), color="black
    ", linestyle="--", label="band edges")
196 plt.plot(S_range.numpy(), (target + H).numpy(), color="black
    ", linestyle="--")
197
198 plt.xlabel("S")
199 plt.ylabel("position")
200 plt.legend()
201 plt.savefig("scatter_check_costs.png")
```

3 Concluding Thoughts

It is clear to see that "Deep Hedging" is a strong strategy when it comes to the hedging of vanilla derivatives in discrete time. The analytical solutions to the hedging problems seen are derived in continuous time environments which do not exist regardless of how high-frequency your trading systems are. This is where the hedging error creeps in for optimal hedging strategies and how neural networks are able to train to the specific problem at hand.

The Whalley and Wilmott [6] solution is understandable and usable in varying market situations. It would be easy for a hedging team to know how often they will adjust their hedging position and therefore how effective this analytical solution will be; ie. the more frequent the repositioning, the closer to continuous time the strategy is, the better performance will be seen. However, as always a neural network is "a black box", if you were to change the number of discrete times you trade at in a given day, the network would need to be retrained to the exact situation once again. On batch sizes (simulated paths) of 1000-2000 and 100 discrete time steps the network took up to 9 minutes to train, for institutional level hedging I would imagine the training would be lengthier and therefore far more computationally costly. The analytical solution does not have this drawback.

For implementing networks like this in the future certain help like that of the sigmoid function to bound $\delta_i \in (0, 1)$ is necessary. Before this implementation the network was producing negative shares in in the lower tails. Tanh as an activation function allows the network to create a non-linear solution which is required to emulate the near normal CDF which is the Black-Scholes delta function. Small tweaks like these are specific to the training of this problem at hand.

Overall it is clear to see that in this simplified version of market dynamics under the constant volatility of the Black-Scholes model, the deep hedging strategies are a solid competitor to those of analytical solutions even with the more unknown, black-box drawback to them. One final plus for deep hedging is the adjustability of the loss function, for the Whalley and Wilmott solution, the entropic risk measure is needed in order to come to an analytical solution, however this may not be the loss function a hedging team cares about. The neural network is able to adjust to any suitable loss function.

References

[1] H. Buehler, L. Gonon, J. Teichmann, and B. Wood, *Deep Hedging*, arXiv preprint arXiv:1802.03042, 2018.

[2] F. Black and M. Scholes, *The Pricing of Options and Corporate Liabilities*, Journal of Political Economy, 81(3), 637–654, 1973.

[3] R. C. Merton, *Theory of Rational Option Pricing*, Bell Journal of Economics and Management Science, 4(1), 141–183, 1973.

[4] K. Hornik, *Approximation Capabilities of Multilayer Feedforward Networks*, Neural Networks, 4(2), 251–257, 1991.

[5] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, arXiv preprint arXiv:1412.6980, 2014.

[6] A. E. Whalley and P. Wilmott, *An Asymptotic Analysis of an Optimal Hedging Model for Option Pricing with Transaction Costs*, Mathematical Finance, 7(3), 307–324, 1997.

[7] M. H. A. Davis, V. G. Panas, and T. Zariphopoulou, *European Option Pricing with Transaction Costs*, SIAM Journal on Control and Optimization, 31(2), 470–493, 1993.

[8] S. D. Hodges and A. Neuberger, *Optimal Replication of Contingent Claims Under Transaction Costs*, Review of Futures Markets, 8(2), 222–239, 1989.